

✓ Fine-Tuning an LLM for Sanskrit↔English (Option 1)

AI/ML Developer Assignment — ImmverseAI (BharatiyaGPT)

This notebook fine-tunes a small open-source instruction-tuned LLM using **QLoRA** to improve Sanskrit↔English translation, explanation, QA, and summarization.

Default / reported model: `Qwen/Qwen2.5-1.5B-Instruct` (ungated, no HF login needed). `meta-llama/Llama-3.2-1B-Instruct` is also supported via the same `BASE_MODEL` variable, but it's gated (needs a free HF account + license acceptance) and took ~18x longer to train under identical config in our testing — see `REPORT.md` section 3/8 for the full comparison.

Runtime: Google Colab, GPU = T4 or L4 (Runtime → Change runtime type → GPU)

Pipeline:

1. Setup & installs
2. Tokenizer inspection (Sanskrit vs English fragmentation)
3. Dataset construction (public parallel corpora → instruction pairs)
4. QLoRA fine-tuning
5. Inference (base vs fine-tuned)
6. Evaluation (BLEU / chrF++, before/after, failure analysis, ranked examples)
7. Save & export adapter

See `README.md` and `REPORT.md` in the repo for setup instructions and the full written report (problem understanding, tradeoffs, failure analysis, what we'd improve with more time).

✓ 1. Setup

```
# Clone the repo and flatten it into /content so later commands can use
# plain relative paths (scripts/..., eval/..., data/...).
!git clone https://github.com/yanil-03/sanskrit-llm.git
!mv sanskrit-llm/* sanskrit-llm/.[!]* . 2>/dev/null || true
!rm -rf sanskrit-llm
```

```
# If you'd rather upload files manually instead of cloning: skip the cell
# above and just upload the `scripts/`, `eval/` folders alongside this
```

```
# notebook in Colab, then continue from here.
```

```
!pip install -q -U transformers accelerate peft trl bitsandbytes datasets sacre
```

```

===== 12.3/12.3 MB 110.4 MB/s eta 0:00:00
===== 394.3/394.3 kB 35.0 MB/s eta 0:00:00
===== 1.0/1.0 MB 67.3 MB/s eta 0:00:00
===== 43.1/43.1 MB 20.7 MB/s eta 0:00:00
===== 559.1/559.1 kB 43.4 MB/s eta 0:00:00
===== 100.8/100.8 kB 7.2 MB/s eta 0:00:00
===== 129.0/129.0 kB 13.7 MB/s eta 0:00:00

```

```

import torch
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("VRAM (GB):", round(torch.cuda.get_device_properties(0).total_memory

```

```

CUDA available: True
GPU: Tesla T4
VRAM (GB): 15.6

```

```

# Hugging Face auth – Llama models are gated; you need a free HF account
# and to accept the license at https://huggingface.co/meta-llama/Llama-3.2-1B-I
# then run:
#   from huggingface_hub import login
#   login() # paste your HF token when prompted
#   BASE_MODEL = "meta-llama/Llama-3.2-1B-Instruct"

# Default used to produce this repo's reported results: ungated, no login needs
BASE_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
BASE_MODEL

```

```
'Qwen/Qwen2.5-1.5B-Instruct'
```

✓ 2. Tokenizer Inspection

Before training, we check how the base tokenizer handles Devanagari/Sanskrit text compared to English of similar content. This is the "Tokenization / Vocabulary" challenge called out in the assignment — Sanskrit's compounding and rich morphology tend to fragment badly under tokenizers trained mostly on Latin-script/English corpora, which affects both training efficiency and the effective context window.

```
!python scripts/inspect_tokenizer.py --model "$BASE_MODEL"
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
config.json: 100% 660/660 [00:00<00:00, 3.75MB/s]
tokenizer_config.json: 100% 7.30k/7.30k [00:00<00:00, 26.6MB/s]
vocab.json: 100% 2.78M/2.78M [00:00<00:00, 82.8MB/s]
merges.txt: 100% 1.67M/1.67M [00:00<00:00, 131MB/s]
tokenizer.json: 100% 7.03M/7.03M [00:00<00:00, 133MB/s]
Tokenizer: Qwen/Qwen2.5-1.5B-Instruct
Vocab size: 151643
```

SA: वसुधैव कुटुम्बकम्।

chars=18 tokens=18 tokens/char=1.00

breakdown: ['àɤ̥μ', 'àɤ̥.', 'àɤ̥g', 'àɤ̥ʂ', 'àɤ̥', 'Ī', 'àɤ̥μ', 'Gàɤ̥k', 'àɤ̥g', 'àɤ̥t']

EN: The whole world is one family.

chars=30 tokens=7 tokens/char=0.23

```
breakdown: ['The', 'Gwhole', 'Gworld', 'Gis', 'Gone', 'Gfamily', '.']
```

--> Sanskrit costs 4.29x more tokens/char than English

SA: धर्मक्षेत्रे कुरुक्षेत्रे समवेता युयुत्सवः।

chars=43 tokens=40 tokens/char=0.93

```
breakdown: ['à¤§', 'à¤°', 'à¥ìà¤', '®', 'à¤«', 'à¥ìà¤', '.', 'à¥ïà¤', '¤', 'à¥
```

EN: On the field of dharma, the field of Kuru, assembled and eager to fight.

chars=72 tokens=19 tokens/char=0.26

```
breakdown: ['On', 'Ġthe', 'Ġfield', 'Ġof', 'Ġd', 'Ġharma', ',', 'Ġthe', 'Ġfield']
```

--> Sanskrit costs 3.53x more tokens/char than English

SA: कर्मण्येवाधिकारस्ते मा फलेषु कदाचन।

chars=35 tokens=33 tokens/char=0.94

breakdown: ['àɪk', 'àɪ', 'àɪàɪ', '®', 'àɪ', 'àɪàɪ', '-', 'àɪàɪ', 'μ', 'àɪ

EN: You have a right to perform your duty, but not to the fruits of action.

chars=71 tokens=17 tokens/char=0.24

```
breakdown: ['You', 'Ghave', 'Ga', 'Gright', 'Gto', 'Gperform', 'Gyour', 'Gduty
```

--> Sanskrit costs 3.94x more tokens/char than English

SA: सत्यमेव जयते।

chars=13 tokens=13 tokens/char=1.00

breakdown: ['à¤', 'à¤¤', 'à¥¤¤', '-', 'à¤®', 'à¥¤¤¤', 'µ', '¤à¤', '¤', 'à¤-']

EN: Truth alone triumphs.

chars=21 tokens=5 tokens/char=0.24

```
breakdown: ['Truth', 'Galone', 'Gtriumph', 's', '.']
```

--> Sanskrit costs 4.20x more tokens/char than English

SA: अहिंसा परमो धर्मः।

chars=18 tokens=18 tokens/char=1.00

breakdown: ['àxh', 'àx¹', 'àx;àx', 'â', 'àx.', 'àx¼', 'Gàx³', 'àx°', 'àx®', 'à

EN: Non-violence is the highest duty.

chars=33 tokens=8 tokens/char=0.24

```
breakdown: ['Non', '-viol', 'ence', 'Gis', 'Gthe', 'Ghighest', 'Gduty', '.']
```

--> Sanskrit costs 4.12x more tokens/char than English

Average Sanskrit/English tokens-per-char ratio: 4.02x

Interpretation: if this ratio is well above 1.0, the tokenizer was not trained with meaningful Devanagari/Sanskrit representation, and

Sanskrit text consumes a disproportionate share of the context window and gradient signal per unit of content relative to English. See REPORT.md for whether we attempted tokenizer adaptation and why.

What to look for: if the average Sanskrit/English tokens-per-character ratio printed above is notably above 1.0 (Qwen2.5 lands around 4x, Llama-3.2 around 2.5x in our runs — see [REPORT.md](#) section 8), Sanskrit text is consuming several times more tokens per unit of content than English. Implications:

- shorter effective context window for Sanskrit passages
- more gradient steps needed to see the same "amount" of Sanskrit content
- a case for tokenizer adaptation (adding Devanagari merges) if we had more time/compute — see REPORT.md section 9 ("What we'd improve").

We did **not** attempt tokenizer adaptation/retraining in this assignment (explicitly marked optional in the brief) given the time budget, but the diagnostic above is what would justify prioritizing it next.

✓ 3. Dataset Construction

We build an instruction-tuning dataset from **rahular/itihasa**, a Sanskrit-English parallel corpus of ~93k sentence pairs derived from the Ramayana and Mahabharata, plus a small curated seed set (Bhagavad Gita verses and well-known subhashitas) reserved for hand-checkable qualitative testing.

Each parallel pair is expanded into **multiple instruction types** (Sanskrit→English, English→Sanskrit, explanation, QA, summarization) using templates, so the model learns to follow varied instructions rather than only one fixed task shape. Full reasoning is in REPORT.md section 2.

```
!python scripts/prepare_data.py --max_pairs 6000 --out_dir data
```

```
Loading rahular/itihasa via parquet (refs/convert/parquet) ...
Parquet load attempt failed (Unable to find 'hf://datasets/rahular/itihasa@~parq
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF

Itihasa/train/0000.parquet: downloading bytes: 27% 4.36M/16.4M [00:01<00:02, 5.
Itihasa/train/0000.parquet: downloading bytes: 82% 13.5M/16.4M [00:01<00:00, 19
Itihasa/train/0000.parquet: reconstructing file: 12% 1.90M/16.4M [00:01<00:07,
Itihasa/train/0000.parquet: reconstructing file: 19% 3.04M/16.4M [00:01<00:05,
Itihasa/train/0000.parquet: reconstructing file: 38% 6.18M/16.4M [00:01<00:01,
Itihasa/train/0000.parquet: reconstructing file: 49% 8.08M/16.4M [00:01<00:01,
Itihasa/train/0000.parquet: reconstructing file: 58% 9.55M/16.4M [00:01<00:00,
```

```

Itihasa/train/0000.parquet: reconstructing file: 76% 12.5M/16.4M [00:02<00:00,
Itihasa/train/0000.parquet: downloading bytes: 100% 16.3M/16.3M [00:02<00:00, 6.
Itihasa/train/0000.parquet: reconstructing file: 100% 16.4M/16.4M [00:02<00:00,

Itihasa/validation/0000.parquet: downloading bytes: 0% 0.00/1.39M [00:00<?, ?B
Itihasa/validation/0000.parquet: downloading bytes: 100% 1.39M/1.39M [00:00<00:0
Itihasa/validation/0000.parquet: reconstructing file: 100% 1.39M/1.39M [00:00<00

Itihasa/test/0000.parquet: downloading bytes: 0% 0.00/2.61M [00:00<?, ?B/s]
Itihasa/test/0000.parquet: downloading bytes: 83% 2.17M/2.61M [00:00<00:00, 3.5
Itihasa/test/0000.parquet: downloading bytes: 100% 2.61M/2.61M [00:00<00:00, 3.1
Itihasa/test/0000.parquet: reconstructing file: 100% 2.61M/2.61M [00:00<00:00, 3
Generating train split: 75162 examples [00:00, 551102.77 examples/s]
Generating validation split: 6149 examples [00:00, 538081.31 examples/s]
Generating test split: 11722 examples [00:00, 617511.29 examples/s]
Loaded 6000 pairs from itihasa (revision fallback).
train=15030 val=835 test=864
Written to data/

```

```

import json

def peek(path, n=3):
    print(f"--- {path} ---")
    with open(path, encoding="utf-8") as f:
        for i, line in enumerate(f):
            if i >= n:
                break
            print(json.loads(line))

peek("data/train.jsonl")
peek("data/test.jsonl")

```

```

--- data/train.jsonl ---
{'instruction': 'Render this English sentence in Sanskrit.', 'input': 'And on th
{'instruction': 'Translate the following Sanskrit text into English.', 'input':
{'instruction': 'Translate the following English text into Sanskrit.', 'input':
--- data/test.jsonl ---
{'instruction': 'Provide a Sanskrit translation of this English sentence.', 'inp
{'instruction': 'Translate the following English text into Sanskrit.', 'input':
{'instruction': 'Translate the following English text into Sanskrit.', 'input':

```

✓ 4. QLoRA Fine-Tuning

4-bit quantized base model + LoRA adapters on attention and MLP projection layers, trained with TRL's `SFTTrainer`. This keeps memory comfortably within a T4's 16GB, and keeps training time within the assignment's suggested 3–6 hour budget (our default config trains in well under that).

Key hyperparameters and the reasoning behind them are documented in REPORT.md section 4/5 (hardware constraints and optimizations).

```
!nvidia-smi --query-gpu=name --format=csv,noheader
```

```
Tesla T4
```

```
!python scripts/train.py \  
  --base_model "$BASE_MODEL" \  
  --train_file data/train.jsonl \  
  --val_file data/val.jsonl \  
  --output_dir outputs/lora-sanskrit \  
  --epochs 2 \  
  --per_device_batch_size 4 \  
  --grad_accum 4 \  
  --max_seq_len 512
```

If you hit an OOM on a T4: lower `--per_device_batch_size` to 2 and raise `--grad_accum` to 8 (same effective batch size, less peak memory), or lower `--max_seq_len` to 384. This tradeoff is discussed in REPORT.md.

For a quick smoke test before committing to a full run, add `--max_train_samples 500` to finish a sanity-check pass in a few minutes.

✓ 5. Inference: Base Model vs Fine-Tuned

We run both the untouched base model and our fine-tuned adapter over the same held-out test set, so we can directly compare before/after outputs.

```
# Base model predictions (no adapter) – this is the "before" baseline  
!python scripts/infer.py \  
  --base_model "$BASE_MODEL" \  
  --test_file data/test.jsonl \  
  --out_file eval/predictions_base.jsonl \  
  --max_new_tokens 150
```

```
# Fine-tuned model predictions – this is "after"  
!python scripts/infer.py \  
  --base_model "$BASE_MODEL" \  
  --adapter outputs/lora-sanskrit \  
  --test_file data/test.jsonl \  
  --max_new_tokens 150
```

```
--out_file eval/predictions.jsonl \
--max_new_tokens 150
```

```
# Quick interactive sanity check on a single example
!python scripts/infer.py \
  --base_model "$BASE_MODEL" \
  --adapter outputs/lora-sanskrit \
  --instruction "Translate the following Sanskrit text into English." \
  --input "वसुधैव कुटुम्बकम्।"
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF

```
model.safetensors: downloading bytes: 0% 2.49M/3.09G [00:00<14:48, 3.47MB/s]
model.safetensors: downloading bytes: 2% 54.3M/3.09G [00:01<00:38, 78.3MB/s, 4
model.safetensors: reconstructing file: 0% 13.7M/3.09G [00:01<04:01, 12.8MB/s,
model.safetensors: reconstructing file: 1% 28.1M/3.09G [00:01<01:48, 28.1MB/s,
model.safetensors: downloading bytes: 4% 124M/3.09G [00:02<00:32, 90.0MB/s, 9.
model.safetensors: downloading bytes: 7% 220M/3.09G [00:02<00:18, 155MB/s, 16.
model.safetensors: downloading bytes: 8% 241M/3.09G [00:03<00:16, 169MB/s, 19.
model.safetensors: downloading bytes: 10% 296M/3.09G [00:03<00:12, 223MB/s, 22.
model.safetensors: reconstructing file: 10% 300M/3.09G [00:03<00:21, 133MB/s, 2
model.safetensors: downloading bytes: 50% 1.53G/3.09G [00:07<00:04, 318MB/s, 1
model.safetensors: reconstructing file: 14% 437M/3.09G [00:07<01:01, 43.0MB/s,
model.safetensors: downloading bytes: 53% 1.62G/3.09G [00:07<00:06, 222MB/s, 1
model.safetensors: downloading bytes: 58% 1.80G/3.09G [00:08<00:06, 190MB/s, 1
model.safetensors: downloading bytes: 63% 1.95G/3.09G [00:09<00:07, 162MB/s, 1
model.safetensors: downloading bytes: 65% 2.01G/3.09G [00:10<00:08, 124MB/s, 1
model.safetensors: downloading bytes: 67% 2.08G/3.09G [00:10<00:07, 139MB/s, 1
model.safetensors: downloading bytes: 76% 2.36G/3.09G [00:12<00:03, 189MB/s, 1
model.safetensors: downloading bytes: 86% 2.66G/3.09G [00:14<00:03, 132MB/s, 1
model.safetensors: downloading bytes: 87% 2.68G/3.09G [00:14<00:03, 129MB/s, 1
model.safetensors: reconstructing file: 60% 1.87G/3.09G [00:15<00:07, 153MB/s,
model.safetensors: reconstructing file: 67% 2.06G/3.09G [00:17<00:08, 116MB/s,
model.safetensors: reconstructing file: 75% 2.33G/3.09G [00:18<00:04, 163MB/s,
model.safetensors: reconstructing file: 82% 2.53G/3.09G [00:18<00:03, 186MB/s,
model.safetensors: reconstructing file: 91% 2.80G/3.09G [00:21<00:02, 142MB/s,
model.safetensors: reconstructing file: 97% 2.99G/3.09G [00:22<00:00, 171MB/s,
model.safetensors: downloading bytes: 100% 2.69G/2.69G [00:22<00:00, 120MB/s, 1
model.safetensors: reconstructing file: 100% 3.09G/3.09G [00:22<00:00, 138MB/s,
Loading weights: 100% 338/338 [00:05<00:00, 59.21it/s]
generation_config.json: 100% 242/242 [00:00<00:00, 1.49MB/s]
```

```
=== PREDICTION ===
O Kautsa, you have been born of Vasudeva.
```

✓ 6. Evaluation

We compute BLEU and chrF++ per task type, generate a before-vs-after comparison table, and run a lightweight heuristic pass to flag likely failure cases (empty outputs,

verbatim-echo, rambling/hallucinated length, degenerate repetition) for manual review.

chrF++ is emphasized over BLEU for Sanskrit because it's character-level and more forgiving of compounding/sandhi variation than word-level n-gram matching — see REPORT.md section 6 for the full discussion of why n-gram metrics are a limited signal here, especially for the non-translation task types (explanation/QA/summarization), which we evaluate mainly qualitatively.

```
!python eval/evaluate.py \  
  --base_predictions eval/predictions_base.jsonl \  
  --finetuned_predictions eval/predictions.jsonl \  
  --out eval/report.md \  
  --n_examples 10
```

Wrote report to eval/report.md

```
{  
  "en2sa": {  
    "n": 45,  
    "bleu": 0.14,  
    "chrF++": 12.48  
  },  
  "explain": {  
    "n": 15,  
    "bleu": 8.14,  
    "chrF++": 25.76  
  },  
  "qa": {  
    "n": 10,  
    "bleu": 1.17,  
    "chrF++": 17.16  
  },  
  "sa2en": {  
    "n": 24,  
    "bleu": 1.62,  
    "chrF++": 22.69  
  },  
  "summary": {  
    "n": 6,  
    "bleu": 2.7,  
    "chrF++": 21.46  
  }  
}
```

```
with open("eval/report.md", encoding="utf-8") as f:  
    print(f.read())
```

- Instruction: Provide an English translation of this Sanskrit sentence.
- Input: `एवमाः सपत्न्यस्ता जग्मुराश्वस्य तां तदा। ददृशुश्चाश्रमे रामं स्वर्गच्युतमिवामरम्॥`
- Reference: `Then those that were co-wives with her, consoled the distressed
- Base model output: `Here is the English translation of the given Sanskrit ve

"Let those who have been married to me be my husband in the future; let him be

This verse appears to be from a poem or song that expresses affection for one'

- Fine-tuned model output: `And that Rāma beheld the wives of Saptarishis in t

****Example 8**** (explain)

- Instruction: Explain the meaning and context of this Sanskrit verse in simpl
- Input: `तस्यैषा लोकनाथस्य धर्मपत्नी यशस्विनी॥ सीता नाम वरारोहा यां त्वं हर्तुमिच्छसि।`
- Reference: `This verse conveys: This exquisitely beautiful and far-famed Sit
- Base model output: `The given Sanskrit verse is:

"यशस्विनी देवास्क्रियाः प्रभवतः शुभे गुणाः।"

This translates to "Yashasvinī devāsikṛitya prabhavati śubhe guṇāḥ."

In simpler terms, it means:

"The goddess Yashasvinī, who is associated with divine qualities, shines bright"

Context: This verse refers to the goddess Yashasvinī, who is considered one of

- Fine-tuned model output: `This verse conveys: And that one is the daughter-i

****Example 9**** (sa2en)

- Instruction: Provide an English translation of this Sanskrit sentence.
- Input: `सात्वमग्निं प्रविश वा स्वयं वा विश दण्डकान्। रज्जु बद्ध्वाथवा कण्ठे नहि तेऽन्यत् परायणम्॥`
- Reference: `Do you enter fire, or dive into Dandaka, or wound a cord round y
- Base model output: `The English translation of the given Sanskrit sentence i

"Fire of Satva or self or VISH Daundika."

This appears to be part of a larger verse or stanza in a text, possibly relate

- Fine-tuned model output: `And the fire will either enter into his body or el

****Example 10**** (sa2en)

- Instruction: Translate the following Sanskrit text into English.
- Input: `हत्वा तु तं भीमबलं विराधं राक्षसं वने। ततः सीतां परिष्वज्य समाश्रय्य च वीर्यवान्॥ अब्रवीद् १`
- Reference: `Having in the forest slain the Rākṣasa Virādha possessed of trem
- Base model output: `Here's the translation of the Sanskrit text into English

"Let me tell you that the demon Bhimbal with his army of demons fought against

"Then Sita approached Lord Rama and Lakshmana, and they both became very happy

"He then said to Bharata, 'I will fight with the enemy.'"

"The forest was darkened by the moonlight, but no one dared to move from there

"At last, he went to the ashram of Shri Raghu, where Ramas were staying."`

- Fine-tuned model output: `And having slain that mighty one of Rākṣasa, Bharat

Per-example ranking (sentence-level chrF++). `evaluate.py` reports corpus-level metrics and a heuristic failure flagger, but doesn't rank individual predictions. The cell below scores every fine-tuned prediction with `sacrebleu.sentence_chrf` and sorts, so the "best"/"worst" examples used in the report are selected automatically and reproducibly, rather than by manually reading through `eval/report.md` (see REPORT.md section 6/7 for the resulting examples and discussion).

```
import sys
sys.path.insert(0, "eval") # so we can import load_jsonl from evaluate.py
from evaluate import load_jsonl
import sacrebleu

def score_and_rank(predictions_file, task_type_filter=None, top_n=5, bottom_n=5):
    """Score every prediction individually (sentence-level chrF++), rank them,
    and print the best- and worst-scoring examples. This is how you find
    genuine successes/failures instead of eyeballing and cherry-picking.

    Uses the same jsonl schema infer.py writes: instruction, input, reference,
    prediction, task_type.
    """
    rows = load_jsonl(predictions_file)
    if task_type_filter:
        rows = [r for r in rows if r.get("task_type") == task_type_filter]
    if not rows:
        print(f"No rows found (task_type_filter={task_type_filter!r}).")
        return []

    def trunc(s, n=max_chars):
        s = s.replace("\n", " ").strip()
        return s if len(s) <= n else s[:n].rstrip() + "..."

    for r in rows:
        pred = r.get("prediction", "").strip()
        ref = r.get("reference", "").strip()
        # sentence_chrf errors on an empty hypothesis/reference pair -> score 0
        if not pred or not ref:
            r["_chrf"] = 0.0
        else:
            r["_chrf"] = sacrebleu.sentence_chrf(pred, [ref], word_order=2).score

    ranked = sorted(rows, key=lambda r: r["_chrf"], reverse=True)

    print(f"=== TOP {top_n} (highest chrF++) – candidates for 'successful cases'")
    for r in ranked[:top_n]:
        print(f"chrF++={r['_chrf']:.1f} task={r['task_type']}")
        print(f"Input: {trunc(r.get('input', ''), 50)}")
        print(f"Reference: {trunc(r.get('reference', ''), 50)}")
```

```
print(f" Prediction: {trunc(r['prediction'])}\n")

print(f"=== BOTTOM {bottom_n} (lowest chrF++) – candidates for 'failure and
for r in ranked[-bottom_n:]:
    print(f"chrF++={r['_chrF']:.1f} task={r['task_type']}")
    print(f" Input:      {trunc(r.get('input', ''))}")
    print(f" Reference: {trunc(r['reference'])}")
    print(f" Prediction: {trunc(r['prediction'])}\n")

return ranked

# Run across all task types together
all_ranked = score_and_rank("eval/predictions.jsonl", top_n=5, bottom_n=5)

# Per-task-type breakdown – cleaner examples per category for the report
for t in ["sa2en", "en2sa", "explain", "qa", "summary"]:
    print(f"\n{' '*20} task_type = {t} {' '*20}")
    score_and_rank("eval/predictions.jsonl", task_type_filter=t, top_n=3, bottc
```

Prediction: O foremost of men, you have not done anything worthy of your sex

chrF++=21.0 task=summary

Input: गजं वा वीक्ष्य सिंहं वा व्याघ्रं वा वनमाश्रिता। नाहारयति संत्रासं बाहू रामस्य संश्रिता॥

Reference: Supporting herself on Rāma's arm, Sītā arrived at the forest, is

Prediction: Do you see a deer or a lion or any other wild animal? Do not app

=== BOTTOM 3 (lowest chrF++) – candidates for 'failure analysis' in REPORT.md

chrF++=20.3 task=summary

Input: मामेवं हि पुरा वीर त्वमेव बहुशोक्तवान्। अनुशिष्याद्धि को नु त्वामपि साक्षाद् बृहस्पतिः॥

Reference: O hero, many a time and oft ere this, you had given me the self-

Prediction: O foremost of men, I have been afflicted with grief on account o

chrF++=19.8 task=summary

Input: भगीरथो हि राजर्षिर्दिव्यं स्यन्दनमास्थितः॥ प्रायादग्रे महाराजस्तं गङ्गा पृष्ठतोऽन्वगात्।

Reference: And stationed on an excellent car that mighty king, the royal sa

Prediction: O Bharata, you have been placed on the throne of Rājarsis; and h

chrF++=16.5 task=summary

Input: प्रसादये त्वां कौसल्ये रचितोऽयं मयाञ्जलिः। वत्सला चानृशंसा च त्वं हि नित्यं परेष्वपि॥

Reference: I deprecate your displeasure. O Kausalyā with joined hands. You

Prediction: O Kausalyā, you have made this beautiful verse for me; and Vatsa

✓ 7. Save / Export

The LoRA adapter (small, a few hundred MB at most) is saved under `outputs/lora-sanskrit/`. Zip and download it, or push it to the Hugging Face Hub, to include it in your submission per the assignment's deliverables checklist.

```
import shutil
shutil.make_archive("lora-sanskrit-adapter", "zip", "outputs/lora-sanskrit")
print("Zipped adapter -> lora-sanskrit-adapter.zip")
```

```
from google.colab import files
files.download("lora-sanskrit-adapter.zip")
```

```
# Optional: push adapter to the Hugging Face Hub instead of downloading a :
# from huggingface_hub import HfApi
# api = HfApi()
# api.create_repo("your-username/sanskrit-llama-lora", exist_ok=True)
# api.upload_folder(folder_path="outputs/lora-sanskrit", repo_id="your-usei
```

Next Steps / What We'd Improve With More Time

See [REPORT.md](#) section 9 for the full list. Highlights:

- Tokenizer adaptation (extend vocab with Devanagari-aware merges) given the fragmentation ratio measured in section 2 above
- Larger, more diverse Sanskrit sources (GRETIL, Digital Corpus of Sanskrit, AI4Bharat) beyond itihasa's epic-poetry register, to generalize past Ramayana/Mahabharata style text
- Human evaluation (native Sanskrit speaker) rather than only automatic metrics, especially for the explanation/QA tasks
- LoRA-vs-full-fine-tune and quantization ablations (bonus items)